

Where the First Root Comes From

A section-by-section guide to `root_origins_study.m`:
the cold lottery, the two-half ladder, the covector handoff, and the basin hunt

M. Casey (Coorbital, `orbit_transfer/DR0_tulip`)

2026-09-16

Abstract

Every seed in the costate library is a previous root. So where did the first one come from? `root_origins_study.m` answers that by doing it: it starts from nothing but two periodic orbits and an engine, and measures the three mechanisms that produce a first root — a cold direct solve where the problem is easy, a ladder down the engine axis that changes chart from collocation to shooting halfway, and a branch-blind hunt for other families. This guide walks the script section by section. Each section is explained three times: as a picture (ELI5), as the reasoning needed to trust it (Intuition), and as the equations and the code (Rigor), with the code snippets sliced verbatim from the script at build time so they cannot drift from it, and the campaign’s measurements called out where they settled a question. It ends with what the script does *not* establish, because the two external reviews of it (FINDINGS 76–77) were mostly about sentences that claimed more than the computation earned. Companions: `seeds_and_solution_route.tex` (what a seed is), `transfer_study_guide.tex` (the gate stack the certifier runs), `process/ROOT_ORIGINS_STUDY.md` (how to run it).

Contents

1	The question, and the honest answer first	1
2	The flow of the script	2
3	§0 — User inputs and the units	4
4	§1 — The two orbits and the two endpoints	5
5	§2 — The cold lottery	6
6	§3 — The direct ladder	7
7	§4 — The handoff: where costates come from	9
8	§5 — The indirect ladder	11
9	§6 — Certification, and adopting the polished root	13
10	§7 — The basin hunt	14
11	§8 — The self-check: gates and outcomes are different things	16

1 The question, and the honest answer first

ELI5

Imagine a library of maps for flying a small spacecraft from one orbit around the Moon to another. Every map in the library was drawn by starting from a neighbouring map and nudging it. Fine — but somebody had to draw the first map with no neighbour to copy. How?

For this particular library the honest answer is: the first map was a gift. A converged transfer for exactly this cell arrived with the `pumpkynPie` package, and every early measurement in the campaign is quoted against it. This script is what you do for every other cell, thrust and orbit pair — which is nearly everything — where there is no gift.

Intuition

Three mechanisms, in the order a campaign uses them, and each one is a *measurement* in the script rather than a claim:

1. **The cold lottery** (§5). Solve the operating point cold, from nothing but the endpoints and a time-of-flight guess, at several mesh densities. The meshes that converge do not agree. That is why continuation is required equipment, not a convenience.
2. **The ladder** (§6–8). Start where the problem is easy — 15 N, a half-day transfer under one turn, converges cold in seconds — and walk the engine down, each rung seeded by the one above. The walk has two halves and a seam: direct collocation down to 0.5 N, then a covector harvest, then multiple shooting the rest of the way.
3. **The basin hunt** (§10). From the root reached, ask the direct solver for other arrival phases. It is branch-blind: it lands wherever its warm start is near. That is a hazard if you assume it returns your family, and a discovery tool if you do not.

Rigor

The problem throughout is the Earth–Moon CR3BP in rotating barycentric nondimensional units ($\mu^* = 0.012150585609624$, $l^* = 389703.26$ km, $t^* = 382981.29$ s), minimum time, low thrust, free t_f , fixed endpoints:

$$\dot{\mathbf{r}} = \mathbf{v}, \quad \dot{\mathbf{v}} = \mathbf{g}(\mathbf{r}, \mathbf{v}) + \frac{T_{\text{nd}} q}{m} \mathbf{u}, \quad \|\mathbf{u}\| = 1, \quad q \in [0, 1], \quad \dot{m} = -\frac{T_{\text{nd}} q}{c_{\text{nd}}},$$

with m a mass *fraction* ($m(0) = 1$), $\mathbf{r}(0), \mathbf{v}(0)$ on the DRO at phase s_D and $\mathbf{r}(t_f), \mathbf{v}(t_f)$ on the tulip at phase s_A . The Hamiltonian, adjoints and the two facts the script leans on:

$$H = 1 + \lambda_r^\top \mathbf{v} + \lambda_v^\top \mathbf{g} + T_{\text{nd}} q \left(\frac{\lambda_v^\top \mathbf{u}}{m} - \frac{\lambda_m}{c_{\text{nd}}} \right), \quad \mathbf{u}^* = -\frac{\lambda_v}{\|\lambda_v\|}, \quad \dot{\lambda}_m = -\frac{\partial H}{\partial m} = -\frac{T_{\text{nd}} q \|\lambda_v\|}{m^2}.$$

Free terminal mass gives $\lambda_m(t_f) = 0$; integrating $\dot{\lambda}_m \leq 0$ backward gives $\lambda_m(t) \geq 0$. Then the coefficient of q in H is $-T_{\text{nd}}(\|\lambda_v\|/m + \lambda_m/c_{\text{nd}}) < 0$ away from a zero primer, and minimising

H selects $q = 1$: **minimum time is all-burn**. The shooting unknown is $z = (\lambda(0), t_f) \in \mathbb{R}^8$ with $H(t_f) = 0$ closing the free-time problem.

2 The flow of the script

Intuition

Two things to notice in the diagram. The data objects change type at §4: above it the trajectory is a discrete collocation solution ($\mathbf{X}$, $\mathbf{U}$, multipliers); below it, it is a shooting root (z plus junction states). That seam is where costates come into existence, and it is the only place they do. And the record $\mathbf{S}$ grows at every box — the script saves after every rung and every probe, so an hour-long run can be read while it is still going, and a stopped run leaves everything it had.

3 §0 — User inputs and the units

ELI5

Everything the run depends on sits in one place at the top: which two orbits, which point on each, what engine we are trying to reach, the two ladders that get there, where to hunt, how long each step may take, and how close is close enough. Change the experiment here and nowhere else.

Intuition

The reference deserves a word. The campaign’s certified root at this cell is stored with the *problem it belongs to* — orbits, phases, engine, mass — and every “vs reference” line in the output is silent unless the live problem matches. Without that, editing the cell would leave a number on screen labelled “the campaign’s answer here” that belongs to somewhere else. (Round-2 review finding; it was exactly that before.)

section 0: the operating point

```
op = struct( ...
    'thrustN', 0.070, ...    % N -- the abstract's Hall thruster
    'ispS',    900, ...     % s
    'm0kg',    150);       % kg
```

section 0: the reference carries its problem

```
ref = struct('tfDays', 17.7976, 'tau', 1.0, 'Np', 7, 'pm', -1, 'sD', 0.0, 'sA', 0.0754, ...
    'thrustN', 0.070, 'ispS', 900, 'm0kg', 150, ...
    'what', 'the fast-family anchor of the 70 mN library (arrived with pumpkynPie)');
```

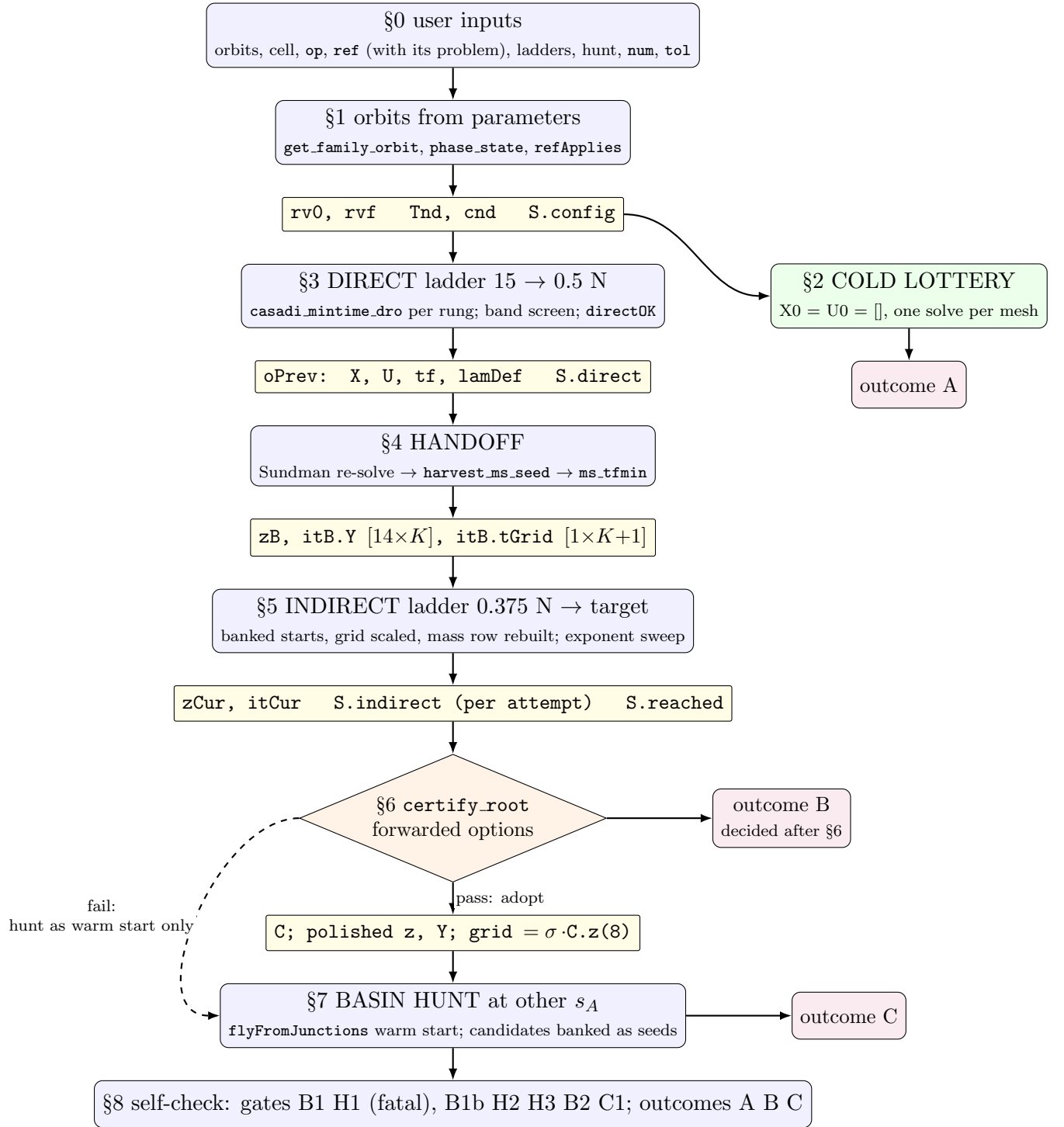


Figure 1: The spine of the script. Yellow boxes are the objects handed from one section to the next; the lottery is a side experiment on the same endpoints; outcome B is decided only after certification, because “reached the target engine” and “holds a certified root there” are different claims.

Rigor

Nondimensionalisation. With $g_0 = 9.80665 \text{ m/s}^2$ and the CR3BP units,

$$T_{\text{nd}} = \frac{T}{m_0} \frac{t^{*2}}{1000 l^*}, \quad c_{\text{nd}} = \frac{I_{sp}}{t^*} g_{0,\text{nd}}, \quad g_{0,\text{nd}} = g_0 \frac{t^{*2}}{1000 l^*},$$

so T_{nd} is the thrust acceleration at unit mass fraction and c_{nd} the exhaust speed, both in ND. The factor 1000 converts l^* from km to m. The tolerance block keeps two kinds of number apart on purpose: what a solver is *asked* for (**Rsolve**, 10^{-11}) and what a gate *accepts* (R , 10^{-8}). A solve that plateaus above its own target but inside the gate is a root.

section 0: the ND conversions

```
g0 = 9.80665*tStar^2/(1000*lStar);
ndT = @(TN, m) (TN/m)*tStar^2/(lStar*1000);      % N -> ND thrust acceleration at m = 1
ndC = @(isp) (isp/tStar)*g0;                      % s -> ND exhaust speed
```

section 0: acceptance thresholds and solve targets

```
tol = struct( ...
    'closure', 1e-7, ... % orbit periodicity
    'seamDeriv', 1e-6, ... % interpolant derivative mismatch across s = 0
    'defect', 1e-8, ... % collocation defect on an accepted rung
    'interp', 1e-8, ... % Hermite-Simpson interpolation residual (NOT in maxDefect)
    'tfSpread', 1e-8, ... % lifted-time continuity across the nodes
    'termErr', 1e-7, ... % terminal boundary residual
    'unit', 1e-8, ... % |u| = 1 on the solved directions
    'thrMin', 1e-3, ... % 1 - throttle: the throttle is FREE, so saturation is CHECKED (see above)
    'R', 1e-8, ... % multiple-shooting residual, inf-norm (acceptance)
    'Rsolve', 1e-11, ... % what the shooting solver is ASKED for (tighter than the gate)
    'tfCluster', 0.02, ... % ND-relative spread within which two t_f are one cluster
    'same', 1e-2); % |t_f - the reference's|/t_f to call the flight times equal
```

Where it breaks / what is assumed

On **thrMin**: the direct solver leaves the throttle free *by design*, so that saturation can be checked rather than assumed. An interior-point solver leaves a barrier slack on every bound — measured 4×10^{-8} to 1.3×10^{-6} at IPOPT tolerance 10^{-7} , and the first full run tripped a 10^{-6} gate on the 15 N rung at $N = 800$ with 1.3×10^{-6} . A real switch takes the throttle to 0. The 10^{-3} gate sits between, and the round-2 reviewer rated it a defensible *seed-compatibility screen*, not a proof of saturation: it cannot see a sustained 0.1% under-throttle or a dip between stations. The all-burn shoot and the certificate downstream are what carry the claim.

4 §1 — The two orbits and the two endpoints

ELI5

Both orbits are rebuilt from their own parameters every run — the DRO from its period, the tulip from its petal count and branch — and a point on each is picked by a fraction of a period. No transfer is loaded anywhere; that is the whole point of the script.

Rigor

`get_family_orbit` starts from a catalogued family seed and refines it to closure; the script checks $\|\mathbf{x}(T) - \mathbf{x}(0)\| < 10^{-7}$ on both. `phase_state` returns a periodic interpolant $s \mapsto \mathbf{x}(s)$ on $[0, 1)$ and its seam mismatch (value and derivative across $s = 0$), which the script gates at 10^{-6} . The endpoints are then $\mathbf{x}_0 = \mathbf{x}_D(s_D)$, $\mathbf{x}_f = \mathbf{x}_A(s_A)$.

section 1: endpoints from phase fractions

```
[stateD, seamD] = phase_state(tD, rvD);  
[stateA, seamA] = phase_state(tT, rvT);  
rv0 = stateD(phase.sD);   rv0 = rv0(1:6);  
rvf = stateA(phase.sA);
```

5 §2 — The cold lottery

ELI5

Ask the big optimiser to solve the real problem with no hints at all: just the two endpoints, a guess at how long the trip takes, and how finely to chop time. Do it three times, chopping time into 400, 800 and 1600 pieces. Every run that finishes says “converged” with tiny errors. They give different trips. Nothing on screen tells you which one you would have wanted.

Intuition

The direct method does not need a seed. With $\mathbf{x}_0 = \mathbf{u}_0 = []$ it builds its own crude guess — states linear between the endpoints, a mass ramp, thrust along the chord — and IPOPT takes it from there. The trouble is never whether it converges; it is *what* it converges to. Minimum-time extremals of this problem come in families, and a cold solve lands in whichever basin its own discretisation happens to sit near. Only the mesh changes between the three runs; the answers change with it. That is the measurement that makes seeding or continuation required equipment on a catalog pair.

What the lottery does *not* show, and the script now says so: that each converged row is a distinct continuous extremal. Changing N changes the finite NLP, its internal guess and its conditioning; a flight-time disagreement alone does not separate “different physical extrema” from those effects. It is mesh sensitivity of the accepted *discrete* solutions — which is enough for the point being made.

Rigor

The transcription is Hermite–Simpson with N intervals on normalised time $s \in [0, 1]$, $d\mathbf{x}/ds = t_f \mathbf{f}(\mathbf{x}, \mathbf{u})$:

$$\mathbf{x}_{k+1} - \mathbf{x}_k - \frac{h}{6}(\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} + \mathbf{f}_{k+1}) = 0, \quad \mathbf{x}_{k+\frac{1}{2}} = \frac{\mathbf{x}_k + \mathbf{x}_{k+1}}{2} + \frac{h}{8}(\mathbf{f}_k - \mathbf{f}_{k+1}),$$

with t_f *lifted* to a per-node variable T_k and continuity $T_{k+1} = T_k$, so each defect touches only its own two nodes and the KKT matrix stays banded (a scalar t_f gives one dense column and the linear solver dies at $N = 400$). The objective is t_f ; the throttle is a free variable in $[0, 1]$.

Every cold solve is judged by the same predicate as every other direct solve in the script (`directOK`, §6); a run that hit its iteration limit is printed as an *iterate* and excluded. On the accepted rows the script reports the spread and a cluster count:

$$n_{\text{cluster}} = |\text{uniquetol}(\{t_f^{(i)}\}, \epsilon)|, \quad \epsilon = \text{tol.tfCluster} \times \min_i t_f^{(i)}, \text{ absolute,}$$

an explicit absolute tolerance tied to the shortest accepted flight time (`uniquetol`'s default scales by the largest element, so a long outlier would regroup the others). A cluster of flight times is not a basin — two roots can share a t_f and one family can span several — it is the cheap first read of the table.

section 2: a genuinely cold solve

```
oM = solveDirect(rv0, rvf, TndOp, cndOp, muStar, N, [], [], cold.tf0, dCold);
[okM, whyM] = directOK(oM, tol);
```

section 2: the statistics on the accepted rows

```
spread = (max([good.tf]) - min([good.tf]))/min([good.tf]);
nCluster = numel(uniquetol([good.tf], tol.tfCluster*min([good.tf]), 'DataScale', 1));
below = nnz([good.nodeRadiusKm] < num.clearKm);
agree = nnz(abs([good.rel]) < tol.same); % 0 when the reference does not apply
```

Measured

Reference at this cell: $t_f = 4.0152 \text{ ND} = 17.798 \text{ d}$. Cold, unconstrained, same solver, same t_f guess (4.0 ND), only N varying.

N	t_f (ND)	vs. reference	min node radius (km)	status
<i>2026-08-03 (logged as altitude; +1737.4 here)</i>				
400	4.3807	+9.1%	6556	converged
800	4.4507	+10.8%	1394	converged, through the Moon
1600	4.7833	+19.1%	6417	converged, 7.8 m worst-interval error
<i>2026-09-16, this script</i>				
400	4.6809	+16.6%	6563	converged
800	4.9909	+24.3%	2243	<i>iterate</i> : iteration limit, excluded
1600	6.4126	+59.7%	1995	converged

Spread 37%, two clusters, none within 1% of the reference. Outcome A: SUPPORTED. “Min node radius” is over the collocation nodes — a screen, not a certified periselene.

6 §3 — The direct ladder

ELI5

Start with an engine so strong the trip takes half a day and barely bends — the optimiser finds that one cold in seconds. Then turn the engine down a little, hand the optimiser yesterday’s trip as a starting point, and solve again. Repeat. Each answer is a small step from the last, so the optimiser never has to search.

Intuition

Three things make the ladder work, and the script measures all three.

The next guess. Between rungs the flight-time guess is scaled by $t_f \leftarrow t_f (T_{\text{prev}}/T)^{0.6}$, an empirical law for this band. Measured rung by rung it predicts the next answer to about 1%, and the script prints the ratio so “starts next to its answer” is a number.

The screen. A rung that lands far from where the scaling predicted has landed somewhere the guess did not describe — and it can look perfect while doing so: converged, defect 10^{-14} , clear of the Moon. Comparing the answer with the guess that produced it is the only cheap thing that catches it. It is a plausibility heuristic, not a branch detector: a sharp turn on the same branch can trip it, and a jump to a nearby-time branch can pass it. It applies to steps only, never to the cold top rung, whose guess has no branch behind it (the true 15 N answer is $41\times$ below the 4.0ND cold guess).

The throttle. The solver leaves it free; every rung is checked to have saturated, on both sides of 1, over nodes and midpoints. Minimum time is all-burn, and a rung whose throttle dipped would not be the problem the shooting solver solves next.

section 3: the inter-rung guess

```
if ~isempty(Tprev), seedTf = seedTf*(Tprev/TN)^dladder.tfExp; end
```

section 3: the band screen, steps only

```
band = oR.tf/seedTf;  
inBand = isempty(Tprev) || (band <= dladder.tfBand && band >= 1/dladder.tfBand);  
clearR = radR >= num.clearKm;  
okR = feasR && clearR && inBand;
```

Rigor

Delta-V falls down the ladder, and the script says why. At fixed exhaust speed the all-burn mass law is $m(t) = 1 - T_{\text{nd}}t/c_{\text{nd}}$, so the propellant fraction used is $1 - m_f = T_{\text{nd}}t_f/c_{\text{nd}}$ and the ideal $\Delta V = c_{\text{nd}} \ln(1/m_f)$. If $t_f \propto T^{-0.6}$ then $T t_f \propto T^{0.4}$ falls as thrust falls: lower thrust, longer trip, less propellant. Measured $15 \rightarrow 0.5 \text{ N}$: $\Delta V \ 4.22 \rightarrow 0.996 \text{ km/s}$, mass used

22.3% $\rightarrow$ 5.8%. (The first draft of the script said the opposite under a table that showed this. Read the table.)

The predicate. Every direct solve — cold, ladder, Sundman re-solve, hunt — is judged by one function so that no section accepts less than another. Required diagnostics default to NaN, not 0, so a missing one fails; the state array must be finite; the throttle history must be finite and inside $[1 - \epsilon, 1 + \epsilon]$. The first failed check is named, and the reasons are reported in an order that never calls an unconverged iterate a branch change.

section 3: ideal delta-V from the mass actually used

```
dv = cnd*log(1/oR.mf)*lStar/tStar; % ideal dV from the mass actually used
```

local function directOK: one predicate for every direct solve

```
fin = @(v) isnumeric(v) && isscalar(v) && isfinite(v) && isreal(v);
thrAll = [];
if isfield(o, 'U') && ~isempty(o.U) && size(o.U, 1) >= 4, thrAll = [thrAll, o.U(4, :)] ; end
if isfield(o, 'Um') && ~isempty(o.Um) && size(o.Um, 1) >= 4, thrAll = [thrAll, o.Um(4, :)] ; end
thrLo = NaN; thrHi = NaN;
if ~isempty(thrAll) && all(isfinite(thrAll)), thrLo = min(thrAll); thrHi = max(thrAll); end
checks = { ...
    'not converged', ~ (isfield(o, 'success') && isscalar(o.success) && o.success); ...
    'non-finite fields', ~ (fin(gvd(o, 'tf', NaN)) && fin(gvd(o, 'mf', NaN)) && fin(gvd(o, 'maxDefect', NaN)) && ...
    fin(gvd(o, 'termErr', NaN)) && fin(gvd(o, 'maxUnit', NaN))); ...
    'states not finite', ~ (isfield(o, 'X') && ~isempty(o.X) && all(isfinite(o.X(:)))); ...
    'collocation defect', ~ (gvd(o, 'maxDefect', NaN) < tol.defect); ...
    'interpolation residual', ~ (fin(gvd(o, 'maxInterp', NaN)) && gvd(o, 'maxInterp', NaN) < tol.interp); ...
    'lifted t_f spread', ~ (fin(gvd(o, 'tfSpread', NaN)) && gvd(o, 'tfSpread', NaN) < tol.tfSpread); ...
    'terminal residual', ~ (gvd(o, 'termErr', NaN) < tol.termErr); ...
    '|u| = 1', ~ (gvd(o, 'maxUnit', NaN) < tol.unit); ...
    'throttle history', ~ (isfinite(thrLo) && isfinite(thrHi)); ...
    'throttle not saturated', ~ (1 - thrLo < tol.thrMin); ...
    'throttle above one', ~ (thrHi - 1 < tol.thrMin); ...
    'mass fraction', ~ (gvd(o, 'mf', NaN) > 0 && gvd(o, 'mf', NaN) <= 1); ...
    't_f not positive', ~ (gvd(o, 'tf', NaN) > 0)};
bad = find([checks{:, 2}], 1);
ok = isempty(bad);
if ok, why = ''; else, why = checks{bad, 1}; end
```

Measured

Rung ratios matter. The first version of the ladder stepped [15, 10, 5, 2, 1, 0.5]. The 1 $\rightarrow$ 0.5 N step (ratio 0.5) returned a converged solution with defect 3.9×10^{-14} , safe periselene and $\|u\| = 1$ at $t_f = 54.7$ d and $\Delta V = 46.8$ km/s: 16 $\times$ its own guess, 94% of the mass burned, a very different angular excursion. The ladder accepted it, and the handoff then ground on a 54-day arc with integrator failures. The shipped rung set steps by about 0.75 and does not jump; the band screen refuses the 16 $\times$ step. On the default run all eleven rungs accept, the worst throttle slack is 1.3×10^{-6} , and t_f goes 0.432 $\rightarrow$ 3.358 d.

7 §4 — The handoff: where costates come from

ELI5

The big optimiser never heard of costates. But to enforce “the trajectory obeys physics” at every step it keeps a set of bookkeeping numbers — how much the trip time would improve if it were allowed to cheat at that step. Those numbers *are* the costates, under another name. Read them off, lay them along the trip at 25 checkpoints, and hand that to the careful solver: it now has a starting trajectory with the steering numbers filled in.

Intuition

Three subtleties turn that from a slogan into working code, and the script checks two of them and says plainly that it cannot check the third. *Station*: a Hermite–Simpson defect multiplier belongs at the segment *midpoint*, not at a node; associating it with a node shifts every costate by half a step and still converges to a slightly wrong answer. *Sign*: the convention depends on how the NLP was written, so it is settled by a vote — the minimum principle requires the applied direction to oppose λ_v at every station, and the sign that satisfies that at the most stations wins (100% on the default run). *Scale*: with t_f as the objective, the multiplier of the time equation must come out $\lambda_t = +1$.

That last row exists only in the Sundman chart, where time is itself a state. The ladder ran in plain time, as the shipped catalog did, so the script re-solves the bottom rung *once* with Sundman on, warm from the plain-time rung, and requires the two to agree in t_f and in node radius before harvesting from it. It buys $\lambda_t = 1.000000$. What a constant row *cannot* see is a half-step station shift: midpoint association is a rule `duals_to_costates` applies, not a thing this number proves.

section 4: the Sundman re-solve must be the same solution

```
dSun = dLad; dSun.sundman = true;
oH8 = solveDirect(rv0, rvf, TndB, cndB, muStar, dladder.N, oPrev.X, oPrev.U, oPrev.tf, dSun);
[feasS, whyS] = directOK(oH8, tol);
% the re-solve is accepted as the SAME solution as the plain-time rung on
% two enforced measurements: t_f agrees to tol.same and the node-sampled
% lunar radius agrees to tol.same -- a re-mesh moves both slightly. An
% aligned trajectory comparison is not made; these two are what is claimed.
sameTf = abs(oH8.tf - oPrev.tf)/oPrev.tf < tol.same;
sameRad = abs(oH8.altMinKm - oPrev.altMinKm)/max(oPrev.altMinKm + rMoonKm, 1) < tol.same;
if ~feasS, sunNote = ['Sundman re-solve refused: ' whyS]; oH8 = [];
elseif ~sameTf, sunNote = sprintf('Sundman re-solve moved t_f by %.1f%%: not the same solution', ...
    100*abs(oH8.tf - oPrev.tf)/oPrev.tf); oH8 = [];
elseif ~sameRad, sunNote = sprintf('Sundman re-solve moved the node radius by %.0f km: not the same
    solution', ...
    abs(oH8.altMinKm - oPrev.altMinKm)); oH8 = [];
```

section 4: harvest, then shoot

```
[seedB, dgB] = harvest_ms_seed(oHarv, num.K);
tH = tic;
[okB, zB, itB] = fenced(pool, @ms_tfmin, 2, num.shootSec + 90, rv0, rvf(1:6), seedB, TndB, cndB, muStar,
    ...
    struct('tolR', tol.Rsolve, 'wallSec', num.shootSec, 'maxIter', 100));
```

Rigor

Sundman and λ_t . With $dt/d\tau = S\kappa(\mathbf{r})$, $\kappa = \rho^p$ (ρ the lunar distance), time is a state $t(\tau)$ with its own defect row. The augmented Hamiltonian carries $\lambda_t \dot{t}$; since nothing depends explicitly on t , $\dot{\lambda}_t = 0$, and the free-time transversality with objective t_f fixes the constant at $\lambda_t = +1$. The eighth multiplier row of the Sundman NLP is a discretisation of that constant, so its value is a free check on the sign and scale of the whole duals-to-costates mapping.

Multiple shooting. Unknowns $p = [\lambda(0); \mathbf{Y}_2; \dots; \mathbf{Y}_K; t_f]$ with $\mathbf{Y}_k \in \mathbb{R}^{14}$ the junction starts ($\mathbf{Y}_1$'s state part is fixed), $7 + 14(K - 1) + 1$ scalars; equations

$$\mathbf{Y}_{k+1} - \Phi_k(\mathbf{Y}_k) = 0 \quad (k = 1..K - 1), \quad \psi(\Phi_K(\mathbf{Y}_K)) = \begin{bmatrix} \mathbf{r}(t_f) - \mathbf{r}_f \\ \mathbf{v}(t_f) - \mathbf{v}_f \\ \lambda_m(t_f) \\ H(t_f) \end{bmatrix} = 0,$$

$14(K - 1) + 8$ of them: square. The Jacobian is block-bidiagonal with one short-segment STM per block, which is why it is conditioned where a single long shot is not. `ms_tfmin` returns `info.Y` as the $14 \times K$ junction *starts* and `info.tGrid` as $K + 1$ times; the endpoint column is never stored, and no consumer reads it (verified in round 2 — the library headers that say $K + 1$ are what is wrong).

Gates. H1: shooting residual under `tol.R`. H2: sign vote $\geq 90\%$. H3 has *three* states — NOT APPLICABLE in plain time (kept out of the pass count), PASS/FAIL in the Sundman chart, where a missing or non-finite λ_t , or a mapping validity flag that is not true, fails. Missing evidence never reads as agreement.

section 4: H3 with three states

```
h3na = isempty(oH8);
if h3na, h3 = false;
else, h3 = isfinite(lamTB) && abs(lamTB - 1) < 1e-2 && islogical(lamTOKB) && isscalar(lamTOKB) &&
    lamTOKB;
```

Measured

Default run: Sundman re-solve within 6×10^{-7} of the plain-time t_f and 4 km in node radius; floor slack 4514 km (the harvested arc does not touch the path constraint the direct solve carried, so its multipliers are costates of the *unconstrained* problem the shooter solves); sign vote 100%; $\lambda_t = 1.000000$ with the mapping's own flag true; $\|R\| = 4.59 \times 10^{-12}$ in *one* iteration. The seed was already a root to the solver's precision.

8 §5 — The indirect ladder

ELI5

Below about half a newton the big optimiser starts losing the thread — the trip winds more and the good answers get harder to find. The careful solver cares less, because it only ever looks at 24 short pieces. So the walk continues with it. And the starting point for the next rung needs no computing at all: the 24 checkpoints of the last answer *are* the starting point,

with their clock stretched to the new guess and the fuel column redone for the new engine.

Intuition

Why no propagation? The banked junction starts are the states and costates at the normalised times k/K . The next rung uses the same K and the same normalised grid, so the seed is the banked array with its time grid scaled to the new t_f guess and its mass row rebuilt from the all-burn law for the new engine. Re-flying the whole arc from $\lambda(0)$ would bring back the amplification multiple shooting exists to avoid, and interpolating a flight would perturb the very numbers the previous solve converged. (Round-2 review; the first version did both.) Why a sweep of exponents? Past a winding wall the next solution has a different angular excursion, and a different exponent is what puts the guess near it. The *fastest accepted* candidate across the sweep is kept. Say what that is: a small search over guesses at each rung, not branch-preserving continuation. It is the campaign's own rule, it is what got the recorded chain past a wall, and it can switch branches — which is why the turn count is printed beside every rung.

Why a stall is a finding. A rung with no accepted candidate is a statement about *this search policy* — these ratios, exponents, budget and seed cap — on this cell. It is not, by itself, a proof that no root exists below. Two in a row stop the walk, and every attempt is recorded with its reason.

section 5: the next seed, without propagating anything

```
Yg = itCur.Y; Yg(1:7, 1) = [rv0(:); 1];
tGs = sigK*tfGuess;
Yg(7, :) = 1 - Tnd*tGs(1:size(Yg, 2))/cnd;
capShoot = min(num.shootSec, left);
[okRun, zt, it] = fenced(pool, @ms_tfmin, 2, capShoot + 90, ...
    rv0, rvf(1:6), struct('tf', tfGuess, 'tGrid', tGs, 'Y', Yg), ...
    Tnd, cnd, muStar, struct('tolR', tol.Rsolve, 'wallSec', capShoot));
```

Rigor

With $\sigma_k = t_k/t_f$ the fixed normalised breakpoints and t_f^g the guess,

$$t_k^{\text{seed}} = \sigma_k t_f^g, \quad \mathbf{Y}_{7,k}^{\text{seed}} = 1 - \frac{T_{\text{nd}} t_k^{\text{seed}}}{c_{\text{nd}}}, \quad t_f^g = t_f^{\text{prev}} \left(\frac{T_{\text{prev}}}{T} \right)^e, \quad e \in \{1.0, 1.3, 1.7, 2.2, 3.0\},$$

with e irrelevant (one attempt) on an Isp-only step. The seed-propellant policy skips a guess with $t_f^g > 0.6 c_{\text{nd}}/T_{\text{nd}}$ — a choice about which seeds to try that keeps 40% of the mass, *not* the exhaustion bound $c_{\text{nd}}/T_{\text{nd}}$ — and records the skip.

Acceptance of a rung is three-fold and fail-closed: the shot converged ($\|R\| < \text{tol.R}$, finite z , $t_f > 0$); the flown witness from z alone reached t_f with positive mass and a finite $n \times 14$ flight; and it landed inside the position *and* velocity gates (100 km, 10 m/s). “Accepted” means a usable continuation seed; physical validation is §9’s job.

The **angular excursion** printed as “turns” is

$$\frac{1}{2\pi} \sum_i |\Delta_i \text{atan2}(y, x - (1 - \mu^*))|$$

along the witness flight in the rotating frame — total variation of the Moon-centred azimuth, so reversals add. It is a diagnostic, not a winding number.

Budget. One deadline per rung (600 s): the remaining time is recomputed before the shoot and before the witness, no stage starts with under 10 s left, and the fence’s cancellation grace is separate from the work budget. A fenced call that returns *not ok* is “timed out or the worker errored” — `run_capped` cannot tell them apart.

section 5: the witness gate, fail-closed

```
reached = okW && isnumeric(yFly) && size(yFly, 2) == 14 && size(yFly, 1) == numel(tw) && ...
    ~isempty(tw) && all(isfinite(yFly(:))) && all(isfinite(tw(:))) && ...
    abs(tw(end) - zt(8)) < 1e-9*max(1, zt(8)) && yFly(end, 7) > 0;
if ~reached, A(end+1) = attempt(tfExp, tfGuess, 'witness flight did not reach t_f', it.normR,
NaN, NaN, toc(tA)); continue, end
mk = norm(yFly(end, 1:3).') - rvf(1:3))*lStar;
mv = norm(yFly(end, 4:6).') - rvf(4:6))*lStar/tStar*1000;
if ~(isfinite(mk) && mk < num.gateKm && isfinite(mv) && mv < num.gateVms)
    A(end+1) = attempt(tfExp, tfGuess, 'flown miss outside the gates', it.normR, mk, mv, toc(tA
)); continue
end
A(end+1) = attempt(tfExp, tfGuess, 'accepted', it.normR, mk, mv, toc(tA));
```

section 5: the angular-excursion diagnostic

```
ang = unwrap(atan2(best.yFly(:, 2), best.yFly(:, 1) - (1 - muStar)));
turns = sum(abs(diff(ang)))/(2*pi);
```

Measured

Default run: 0.375 → 0.12 N at $I_{sp} = 1710$ s (7 rungs), the Isp stage 1710 → 1400 → 1150 → 900 s at 0.12 N (3 rungs in 3 s, t_f falling 11.119 → 10.606 d), then 0.11 and 0.10 N: every one of the five exponents *unconverged*, no fence event, no seed-policy skip. Turns climbed 0.81 → 1.31. Worst accepted witness miss 1.8×10^{-4} km. Four routes were tried on this cell; the stall moved from 0.12 to 0.105 to 0.11 N with finer ratios and the Isp stage, and the campaign reached 0.09 N only from the sheet’s fastest 0.5 N cell. Where you start the deep walk is a choice, not a detail.

9 §6 — Certification, and adopting the polished root

ELI5

Now hand the trip to the inspector. It re-solves it to higher precision, flies it from the initial numbers alone, checks every condition the theory demands, asks a second, independent solver to agree, and tests that it is a true minimum and not a saddle. If it passes, the inspector’s re-solved version is the one we keep.

Intuition

Two things the round-2 review caught here, both mine.

Forwarding. The certifier reads the initial mass, the arrival gates and the lunar floor from *its own options*, not from the problem struct. Putting them in the problem struct did nothing; editing them in §0 would have silently left certification on defaults. They are forwarded and printed on the certify line.

The grid. The certifier polishes and returns the polished z and junction states, but no time grid. The shooter keeps the *normalised* breakpoints fixed and re-solves t_f , so the polished states belong to the polished t_f . I had paired them with the old grid. The certificate was valid; the record and the hunt’s warm start were inconsistent by the polish’s change in t_f on every segment — dormant on the default run only because the walk’s own root was already at the polish target.

section 6: forward what certify_root reads, and the poolless policy

```
certOpts = struct('pool', pool, 'wallSec', num.certSec, 'sA', phase.sA, 'sD', phase.sD, ...
    'm0kg', op.m0kg, 'gateKm', num.gateKm, 'gateVms', num.gateVms, 'moonKmMin', num.clearKm,
    ...
    'allowUnfenced', num.allowUnfenced);
if isempty(pool) && ~num.allowUnfenced
    % the poolless policy, stated in section 0: no fence, no certificate
    C = struct('ok', false, 'reason', 'not certified: no parallel pool and num.allowUnfenced is false', ...
        'tfDays', day(zCur(8)), 'flyKm', NaN, 'flyVms', NaN, 'z', zCur(:), 'Y', itCur.Y);
else
    C = certify_root(seedR, rv0, rvf, B, certOpts);
```

section 6: adopt the polished root WITH its grid

```
tfPre = zCur(8);
zCur = C.z(:);
itCur.Y = C.Y; itCur.tGrid = sigK*zCur(8); itCur.normR = gvd(C, 'normR', NaN);
```

Rigor

$t_k^{\text{cert}} = \sigma_k C.z_8$ with σ the normalised grid preserved from the walk; the shapes are asserted before adoption and the relative change in t_f is printed (it reads 0 when the walk’s root was already polished, which is the honest number). Without a parallel pool the certifier refuses to run unfenced unless told to; the script’s policy knob `num.allowUnfenced` (default false) makes that explicit and records “not certified: no pool” rather than aborting after an hour. The gate stack itself — polish, flown arrival in position and velocity, pointwise PMP checks, the foreign-solver witness, the free-time conjugate test, the hypothesis gates and the H6 validity condition — is `transfer_study_guide.tex`’s subject. Here only its verdict is used, and outcome B is decided *after* it: SUPPORTED means a certified root at the target engine; “reached numerically” is its own state.

section 6: outcome B, decided after certification

```
if b2 && C.ok, outcomeB = sprintf('SUPPORTED: a certified root at the target engine, walked from a
    cold %g N start', dladder.rungs(1, 1));
elseif b2, outcomeB = 'REACHED NUMERICALLY: the target engine, but the root there did not certify
    ';
```

```

elseif iladder.run, outcomeB = sprintf('PARTIAL: reached %g N / %g s of %g N / %g s%s', Tcur, ispCur, op.
    thrustN, op.ispS, ...
    tern(C.ok, ', certified there', ', not certified there'));
else,
    outcomeB = sprintf('PARTIAL: direct ladder only, %g N / %g s%s', Tcur, ispCur, ...
    tern(C.ok, ', certified there', ', not certified there'));
end

```

10 §7 — The basin hunt

ELI5

Now that we hold one trip, go looking for others. Ask the big optimiser for a trip to a *different* arrival point, starting from the trip we have. It does not know which family it came from, so it may hand back another — sometimes a faster one. That is how three of the five families in the library were found.

Intuition

What a probe measures, exactly — and this is the sentence the round-1 review changed. Each probe is a *different* boundary-value problem (the arrival state moved), so its t_f is not comparable to the source's as a basin test: one family's t_f varies with phase by far more than a few percent, and two families can share a t_f . A probe yields a *candidate*: a converged transfer at phase s_A , some percent faster or slower than the source *at its own phase*. Turning a candidate into a basin verdict needs a same-phase baseline — continue the held family to s_A with `arclength_arrival` and compare states and costates there. That is what FINDINGS 61, 63 and 68 did. Every converged probe is banked as a harvested seed so that it can be.

Rigor

The warm start needs dense samples on $N + 1$ nodes, which the banked junctions alone are not, so here — and only here — the source root is re-flown *segment by segment* from its junction starts. No segment is longer than $1/K$ of the arc; at each interior seam the *banked start* of the next segment is kept and the previous segment's propagated endpoint dropped, so a resample at a junction recovers the banked state exactly; the worst seam mismatch is returned in km rather than hidden (2.7×10^{-8} on the default run). Duplicate time samples are removed, the primer norm is required finite and positive, and the direction guess is $\mathbf{u}_0 = -\lambda_v / \|\lambda_v\|$ with throttle 1.

```

local function flyFromJunctions: segment by segment, the banked start wins the seam

for k = 1:K
    dt = tG(k+1) - tG(k);
    [ts, ys] = pumpkyn.cr3bp.tfMinProp(dt, Y(:, k), Tnd, cnd, mu);
    assert(isnumeric(ys) && size(ys, 2) == 14 && size(ys, 1) == numel(ts) && all(isfinite(ys(:))) && ...
        abs(ts(end) - dt) < 1e-9*max(1, dt), 'flyFromJunctions:segment', 'segment %d did not complete',
        k);
    if k < K
        seamKm = max(seamKm, norm(ys(end, 1:3).' - Y(1:3, k+1))*lStarKm);
        ts = ts(1:end-1); ys = ys(1:end-1, :); % the banked start of k+1 wins the seam
    end
    tj = [tj; tG(k) + ts(:)];
end

```

```

    yj = [yj; ys];
end

```

section 7: the direct warm start from the re-flown source

```

sN = linspace(0, tjH(end), dladder.N + 1);
X0h = interp1(tjH, yjH(:, 1:7), sN, 'spline').';
LVh = interp1(tjH, yjH(:, 11:13), sN, 'spline');
nLV = vecnorm(LVh, 2, 2);
assert(all(isfinite(nLV)) && all(nLV > 1e-12), 'the primer vanishes or is not finite on the source
flight');
U0h = [(-LVh ./ nLV).'; ones(1, dladder.N + 1)];

```

Rigor

Three counts are kept apart for each probe: did the solver converge, was the solution accepted (directOK and the floor), was a seed harvested from it. An accepted solution whose multipliers did not come back keeps its numbers ($\mathbf{X}, \mathbf{U}, t_f$) so “anchor it” stays possible by re-solving. A capped probe is reported as “solver did not converge” and nothing more — a lottery ticket lands on either side of its budget from run to run (the $s_A = 0.7837$ probe converged in 61 s on one run and hit its 300 s CPU cap on the next), and the script no longer reads that as anything about basins.

section 7: the reading for each probe, in order of precedence

```

if ~oH.success, rd = ['solver did not converge: ' oH.ipoptStatus];
elseif ~feasH, rd = ['converged but refused: ' whyH];
elseif ~clearH, rd = sprintf('converged but refused: node radius %.0f km under the %g km
floor', radH, num.clearKm);
elseif rel < 0, rd = sprintf('candidate, %.1f%% FASTER than the source at its own phase --
worth a same-phase baseline', -100*rel);
else, rd = sprintf('candidate, %.1f%% slower than the source at its own phase',
100*rel);
end
if okH && ~harvOK, rd = [rd ' (NO seed harvested: multipliers unavailable; direct solution kept
)']; end

```

Measured

Default run, from the certified 0.12 N root at $s_A = 0.0754$ ($t_f = 10.606$ d): $s_A = 0.3254 \rightarrow 11.240$ d (+6.0%, candidate); $s_A = 0.5754 \rightarrow 9.081$ d (−14.4%, candidate, banked); $s_A = 0.7837 \rightarrow$ CPU cap. Outcome C: CANDIDATES, 2 accepted, 1 faster, 2 banked; basin identity not established here.

11 §8 — The self-check: gates and outcomes are different things

ELI5

At the end the script grades itself twice. First: did the machinery work? If the ladder produced nothing or the harvest could not be shot to a root, nothing above can be believed and the script fails. Second: what did the three experiments *show*? A search that found nothing is a result, not a broken machine.

Rigor

Gates B1 (the direct ladder produced a rung) and H1 (the harvest shot to a root) are fatal. B1b, H2, H3, B2 and C1 are reported, never fatal; H3 is removed from the table when not applicable. Outcomes take the values SUPPORTED, PARTIAL, REACHED NUMERICALLY, CANDIDATES, NOT OBSERVED, INCONCLUSIVE. The whole record — configuration, tolerances, every rung and probe with its reason, the certificate, the value-only certifier options, the gate table and the outcomes — is saved, and a failed save warns rather than pretending.

section 8: the gate table

```
gateStatus = { ...
  'B1 direct ladder produced a rung', b1; ...
  'B1b direct ladder finished',      b1b; ...
  'H1 handoff shot to a root',      h1; ...
  'H2 harvest sign vote',           h2; ...
  'H3 harvest lambda_t',            h3; ...
  'B2 target engine reached',       b2; ...
  'C1 certified',                   C.ok};
if h3na, gateStatus(strcmp(gateStatus(:, 1), 'H3', 2), :) = []; end % not applicable is not a verdict
outcomes = {'A cold lottery', outcomeA; 'B ladder', outcomeB; 'C basin hunt', outcomeC};
```

12 What the script does not establish

Where it breaks / what is assumed

- That the lottery's converged rows are distinct *continuous* extremals. It shows mesh sensitivity of accepted discrete solutions.
- That the throttle is exactly saturated. The 10^{-3} gate is a seed-compatibility screen; the all-burn shoot and the certificate carry the claim.
- That the stall at 0.11 N is a property of the problem. It is a property of this search policy on this cell; arclength steps in thrust, rank and sensitivity of the shooting Jacobian and a budget study would be needed to say more.
- That any hunt candidate is another family. That needs a same-phase baseline.
- That a certified root at 0.12 N says anything about 0.07 N. Outcome B on the default run is PARTIAL, and says so.
- Anything about a capped probe except that it was capped.

Every one of these was, at some point, a sentence in an earlier draft that claimed more. FINDINGS 76 and 77 record which reviewer caught which.

13 Summary

cold lottery	one cold solve per mesh; the accepted rows disagree by 37% and none matches the reference; continuation is required equipment
direct ladder	$15 \rightarrow 0.5\text{ N}$, guess $\propto T^{-0.6}$ to 1%; band screen on steps; throttle checked both sides; ΔV falls because $Tt_f \propto T^{0.4}$
handoff	Sundman re-solve of the bottom rung so λ_t exists; midpoint stations, sign vote, $\lambda_t = 1.000000$; one shooting iteration
indirect ladder	banked starts are the next seed, grid scaled, mass row rebuilt; exponent sweep is a search; per-attempt records; a stall is a policy finding
certify	options forwarded; polished root adopted with $t_k = \sigma_k C.z_8$; poolless policy explicit; outcome B decided after
basin hunt	re-fly from junctions, banked start wins the seam; candidates, not basins; seeds banked; capped is capped
self-check	fatal gates B1, H1; the rest reported; outcomes in their own words
